雞兔同籠問題(佔分 10 分)

問題描述：

一個主人在大籠子中養了雞和兔子兩種動物，假設雞全部都是 2 隻腳以及兔子全部都是 4 隻腳，在已知主人計算出籠子中雞和兔子合計總數量以及籠子中雞和兔子合計總腳數的狀況下，請分別找出雞的數量和兔子的數量。但是，請注意主人有時可能會算錯總數量或總腳數，導致本問題無解。

輸入說明：

輸入資料共有五列，每一列有兩個整數，以空格隔開，第一個整數為籠子中動物的合計總數量，第二個整數為籠子中動物的合計總腳數。

輸出說明：

輸出資料共有五列，根據每列輸入資料分別計算雞的數量和兔子的數量，共兩個整數，以空格隔開，若某列輸入資料無解，則本列輸出”no solution”。

範例說明：

輸入：

```
252 872
252 504
252 685
567 1234
678 1234
```

輸出：

```
68 184
252 0
no solution
517 50
no solution
```

DNA序列比對(佔分10分)

問題描述：

根據維基百科的介紹，去氧核糖核酸（DNA）是一種生物大分子，負責攜帶遺傳指令，指導生物的生長發育和生命功能運作。每個細胞核內的 DNA 序列包含約 30 億個基本單位（由 A、T、G、C 四種氮鹼基組成），這些序列控制生物體內蛋白質的生成。由於每個人的 DNA 序列各不相同，通過 DNA 檢測可以區分出不同個體之間的 DNA 結構差異。輸入 N 個人的 DNA 序列，請計算出任意 2 人之間所有排列組合中，每一組合 DNA 序列最大的重複百分比，最後求所有組合之平均值 P%，重複的氮鹼基可以不必是連續的，但它們在 DNA 序列中的相對順序必須一致。

輸入說明：

第 1 列有 1 個數字 N，表示有 N 個人。

第 2 列到第 N+1 列中，每一列的 DNA 序列其氮鹼基長度相同，最短為 4 個氮鹼基，最長為 1024 個氮鹼基。

輸出說明：

輸出 1 個數字 P (0.00≤P≤100.00，四捨五入到小數點以下兩位)，表示任意 2 人之間所有排列組合，其 DNA 序列中最大重複百分比的平均值 P%。

範例 1：

輸入：

```
2
ATCGATCG
ATCGATCC
```

輸出：

```
87.50
```

範例 2：

輸入：

```
3
GATC
ATCT
TCGC
```

輸出：

```
58.33
```

影像之卷積運算(佔分 10 分)

問題描述：

黑白影像通常可用一 N*N 矩陣來儲存其資訊，其值為整數。如下 5*5 矩陣就是某黑白影像之資訊。

```
1 1 1 0 0
0 1 1 1 0
0 0 1 1 1
0 0 1 1 0
0 1 1 0 0
```

卷積運算是深度學習中非常重要的概念。假設今天我們要以 3*3 矩陣當內核(或稱 filter)，來對黑白影像做卷積運算後，可得到一 feature map. Feature map 也是一矩陣。例如在下例中，A 為一影像資訊、B 為內核資訊、F 為運算後 Feature map 資訊。

$$A = \begin{bmatrix} a11 & a12 & a13 & a14 & a15 \\ a21 & a22 & a23 & a24 & a25 \\ a31 & a32 & a33 & a34 & a35 \\ a41 & a42 & a43 & a44 & a45 \\ a51 & a52 & a53 & a54 & a55 \end{bmatrix}$$

$$B = \begin{bmatrix} b11 & b12 & b13 \\ b21 & b22 & b23 \\ b31 & b32 & b33 \end{bmatrix}$$

$$F = \begin{bmatrix} f11 & f12 & f13 \\ f21 & f22 & f23 \\ f31 & f32 & f33 \end{bmatrix}$$

在某卷積運算環境是把內核 B 中 b11 對齊 A 中元素後，在 A 中若能得到 3*3 子矩陣 A'，則將 A'和 B 兩矩陣做內積計算。例如把內核 B 中 b11 對齊 A 中 a11 後，得

f11 = b11*a11+ b12*a12+ b13*a13+ b21*a21 + b22*a22+ b23*a23 + b31*a31+ b32*a32+ b33*a33

$$A = \begin{bmatrix} a11 & a12 & a13 & a14 & a15 \\ a21 & a22 & a23 & a24 & a25 \\ a31 & a32 & a33 & a34 & a35 \\ a41 & a42 & a43 & a44 & a45 \\ a51 & a52 & a53 & a54 & a55 \end{bmatrix} \quad B = \begin{bmatrix} b11 & b12 & b13 \\ b21 & b22 & b23 \\ b31 & b32 & b33 \end{bmatrix}$$

把內核 B 中 b11 對齊 A 中 a12 後，得

f12 = b11*a12+ b12*a13+ b13*a14+ b21*a22+ b22*a23+ b23*a24+ b31*a32+ b32*a33+ b33*a34

$$A = \begin{bmatrix} a11 & a12 & a13 & a14 & a15 \\ a21 & a22 & a23 & a24 & a25 \\ a31 & a32 & a33 & a34 & a35 \\ a41 & a42 & a43 & a44 & a45 \\ a51 & a52 & a53 & a54 & a55 \end{bmatrix} \quad B = \begin{bmatrix} b11 & b12 & b13 \\ b21 & b22 & b23 \\ b31 & b32 & b33 \end{bmatrix}$$

f13、f21、f22、f23、f31、f32、f33 可依此類推而得到。在此計算不會有 f14，因為把內核 B 中 b11 對齊 A 中 a14，在 A 中不能得到 3*3 子矩陣。同樣情形，也不會有 f15、f24、 f25、f41、f42…。 因此若

$$A=\begin{bmatrix}1&1&1&1&0\\0&1&1&1&0\\0&0&1&1&1\\0&0&1&1&0\\0&1&1&0&0\end{bmatrix}\quad B=\begin{bmatrix}1&0&1\\0&1&0\\1&0&1\end{bmatrix}$$ 則 F=$\begin{bmatrix}4&3&4\\2&4&3\\2&3&4\end{bmatrix}$，F 為一 3*3 矩陣。

若

$$A=\begin{bmatrix}1&1&1&1\\0&1&1&1\\0&0&1&1\\0&0&1&1\end{bmatrix}\quad B=\begin{bmatrix}1&0&1\\0&1&0\\1&0&1\end{bmatrix}$$ 則 F=$\begin{bmatrix}4&3\\2&4\end{bmatrix}$，F 為一 2*2 矩陣。

輸入說明：

第 1 列為 N*N 矩陣 N 之值，其中 3≤N≤1000

第 2 列開始 N 列為圖片 A 之內容，每列中元素以空白隔開。

接下來 3 列為內核 B 3*3 矩陣之內容，每列中元素以空白隔開。

輸出說明：

印出 feature map 中由左上到右下對角線中之元素於一列中，各元素以空白隔開。

範例說明：

輸入：

```
5
1 1 1 0 0
0 1 1 1 0
0 0 1 1 1
0 0 1 1 0
0 1 1 0 0
1 0 1
0 1 0
1 0 1
```

輸出：

```
4 4 4
```

文章餘弦相似度計算(佔分 10 分)

問題描述：

在某一資料庫中存有英文文章，這些文章內容由英文單字、數字和標點符號組合而成，標點符號只有三種，分別是":"(冒號)、"',"(逗號) 和 "."(句號)。數字視為一英文單字，且不考慮數字有小數點的存在。文章中單字以空白隔開，但單字和標點符號可能連在一起。每篇文章內單字和標點符號總數會小於 1 萬，且可能有好幾個段落的內容。

今天我們要計算兩文章的相似程度，首先我們會將每篇文章中出現的英文單字依出現順序列出，並統計此單字在此文章出現幾次，英文大小寫視為相同，並將標點符號捨棄、 若單字組成字母不一樣就視為不同單字，如 apple 和 apples 為不同單字。 因此在 A 文章內容為；"I wonder how, I wonder why." 則 "i" 出現 2 次、"wonder" 出現 2 次、"how" 出現 1 次、"why"出現 1 次。

B 文章內容為；"I wonder, why I am so afraid. I wonder, I wonder." 則 "i" 出現 4 次、"wonder"出現 3 次，"why" 出現 1 次、"am" 出現 1 次、"so" 出現 1 次、"afraid" 出現 1 次

接下來我們會依據單字出現在兩篇文章次數總和，由多至少來選取 N 個元素當軸組成一向量空間。 若有多個單字有相同出現次數和，則會先選在 A 文章先出現的單字，次一個換選在 B 文章先出現的單字，再次一個回到 A 文章選下一個出現的…， 依此交互在 AB 文章中選出單字。若有一文章已無單字可選，則剩下來的單字皆從另一篇文章中選出。若兩篇文章都已無單字可選，則將剩下的軸皆以 0 帶入兩文章向量中。例如在上面例子中， 若 N=6 則我們會依序選取"I"、"wonder"、"why",、"how"、"am"、 "so" 六元素當軸。

此時我們可將每篇文章用選取之軸來組成高維向量空間呈現，每軸之值為該單字在該文章出現的次數。此時 A 文章 = (2, 2, 1, 1, 0, 0)

B 文章 = (4, 3, 1, 0, 1, 1)

最後可用餘弦相似度(cosine similarity)=

$$\frac{A\cdot B}{||A||||B||}=\frac{\sum_{i=1}^{n}(A_i * B_i)}{\sqrt{\sum_{i=1}^{n}A_i{}^2}*\sqrt{\sum_{i=1}^{n}B_i{}^2}}$$，(A_i 和 B_i 分別代表向量 A 和 B 的各分量)

$$=\frac{(2*4+2*3+1*1+1*0+0*1+0*1)}{\sqrt{2^2+2^2+1^2+1^2+0^2+0^2}*\sqrt{4^2+3^2+1^2+0^2+1^2+1^2}}$$

=0.9

輸入說明：

第 1 列為向量維度 N 之值， 其中 0<N<1000

第 2 列開始為 A 文章之內容，以*標示結束。*前可能有空白存在。

下列開始為 B 文章之內容，以*標示結束。*前可能有空白存在。

輸出說明：

印出 A 和 B 之餘弦相似度(cosine similarity)，四捨五入到小數點後第二位。

範例說明：

輸入：

```
6
I wonder how, I wonder why. *
I wonder, why I am so afraid.
I wonder, I wonder. *
```

輸出：

```
0.90
```

長實數相乘後轉為二進位(佔分 10 分)

問題描述：

寫一個C程式，輸入2個最長為1024個位數的十進位正實數（有小數點以下數字，最長10位），將相乘後的結果轉換為二進位（小數點以下數字固定10位，第10位以後位數捨去）

輸入說明：

第 1 與第 2 列分別有一個最長為 1024 個位數的十進位正實數（有小數點以下數字，最長 10 位）

輸出說明：

輸出 1 個數字，表示輸入兩個數字相乘後的二進位結果（小數點以下數字固定 10 位，第 10 位以後位數捨去）

範例 1：

輸入：

12.34

56.78

輸出：

1010111100.1010101001

範例 2：

輸入：

12345678901234567890123456789 0.1

98765432109876543210987654321 0.2

輸出：

100110110110011001100000100011000001100000000000100000011111010010111001101100111111100000000000010010001110100101111100111101010110000100111101110010011110111011111010000110111001100101001110110011111.0000010100

找最遠格子填滿棋盤(佔分 10 分)

問題描述：

一個 m×n 的二維棋盤格子，座標以 row major 方式排列([1,1]、[1,2]、…、[1,n]、[2,1]、[2,2]、…、[m,n-2]、[m,n-1]、[m,n])，其中部分格子為路障(X)代表無法通行。從某一個起點格子(標記編號 1)開始走，每次尋找離起點格子最遠的未標記格子為新的起點(標記編號加 1)，被標記後的格子也會形成路障。每個格子都只能往上、下、左、或右走，但是路障不能通行(需繞路走)，「離最遠」代表從起點格子開始走到此未標記格子的最短距離格子數為最多者，如果有兩個以上相同距離的最遠格子，從中選擇列座標較大者為新的起點，若列座標相同的情況則選擇行座標較大者。以圖一的 5×5 二維棋盤格子為例，[1,1]座標起點(標記編號 1)的最遠未標記格子為[5,5]座標(最短距離 8，標記編號 2 為新起點)，編號 2 的下一個最遠未標記格子為[2,1]座標(標記編號 3，最短距離 7 有兩個格子，但[1,2]座標的列座標較小未獲選)，編號 3 的下一個最遠未標記格子為[3,5]座標(標記編號 4，最短距離 7，因為[3,4]為路障需繞遠路)。圖一右邊為最後填滿的結果，其中[4,2]座標格子無法標記編號，因為編號 21 的[3,3]座標無法走到[4,2]座標格子。

1				
3	X			
			X	4
X				2

→

1	12	16	10	6
3	X	20	14	8
5	18	21	X	4
13		19	17	11
X	9	15	7	2

圖一、5×5 二維棋盤格子且起點為[1,1]座標之例子

輸入說明：

輸入資料第 1 列為兩個整數 m 和 n，以空格隔開，整數值範圍為 5 ≤ m,n ≤ 15，代表格子數量為 m×n 的二維棋盤格子。第 2 列為兩個整數 x 和 y，以空格隔開，其中 1 ≤ x ≤ m 且 1 ≤y ≤ n，代表標記編號 1 的起點格子為[x,y]座標。第 3 列之後的列數不固定(0 到多列)，每列皆為兩個任意整數 i 和 j，以空格隔開，代表[i,j]座標為路障格子，路障數量不固定。座標以 row major 方式排列([1,1]、[1,2]、…、[1,n]、[2,1]、[2,2]、…、[m,n-2]、[m,n-1]、[m,n])。最後輸入一列 -1 -1 的兩個整數代表輸入結束。

輸出說明：

總共輸出 m×n 個整數，以空格隔開，並以 row major 的順序列出，其數值顯示[1,1]、[1,2]、…、[1,n]、[2,1]、[2,2]、…、[m,n-2]、[m,n-1]、[m,n]每個格子的標記編號，路障格子和無法標記格子(無法走到)的編號均顯示為整數 0。

範例說明：

輸入：

```
5 5
1 1
2 2
3 4
5 1
-1 -1
```

輸出：

```
1 12 16 10 6 3 0 20 14 8 5 18 21 0 4 13 0 19 17 11 0 9 15 7 2
```

XOR 數字(佔分 10 分)

問題描述：

在數位邏輯中的 XOR 運算，對於 $c = a$ XOR b，若 $a = 1, b = 0$，或 $a = 0, b = 1$，則 c 之值為 1。反之，若 a 與 b 同時為 1，或同時為 0，則 c 之值為 0。給定一個由 N 個 0 或 1 組成的二進位數 $S_1 = a_1a_2a_3…a_N$，請產生另一個新的二進位數 $S_2 = b_1b_2b_3…b_N$，其中 $b_i = a_{i-1}$ XOR a_{i+1}。對於最左邊的位元 b_1 而言，$b_1 = a_N$ XOR a_2，而對最右邊的位元 b_N 而言，$b_N = a_{N-1}$ XOR a_1。

舉例來說，若 S_1 = 1010101001，則 $b_1 = a_{10}$ XOR a_2 = 1 XOR 0 = 1、$b_5 = a_4$ XOR a_6 = 0 XOR 0 = 0、$b_6 = a_5$ XOR a_7 = 1 XOR 1 = 0、$b_{10} = a_9$ XOR a_1 = 0 XOR 1 = 1。最後可得到 S_2 = 1000000111。

輸入說明：

第 1 列有一個數字 N，表示二進位數 S_1 的位元個數，N 之值不超過 50。

第 2 列有 N 個數字，表示 S_1 的 N 個位元值。

輸出說明：

輸出為一列，有 N 個數字，表示二進位數 S_2 的 N 個位元值。

範例說明：

輸入：

```
10
1 0 1 0 1 0 1 0 0 1
```

輸出：

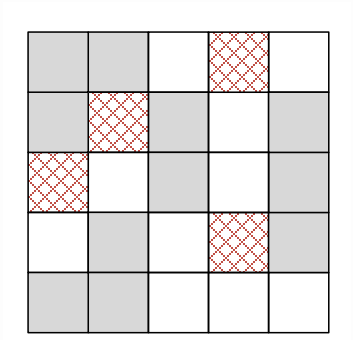
```
1 0 0 0 0 0 0 1 1 1
```

地磚區塊(佔分 10 分)

問題描述：

家中的客廳地板是一個正方形區域鋪了 NxN 塊正方形地磚。這些地磚有些是污損的，有些是乾淨的，而乾淨的地磚有灰色與白色之分。兩塊乾淨且同色的地磚若相鄰有一條共同邊，我們稱這兩塊地磚是屬於同一區塊。因此若灰色地磚 A 與灰色地磚 B 屬於同一區塊，且 B 與灰色地磚 C 亦屬於同一區塊，則 A、B、C 三塊地磚形成一個灰色區塊，依此類推。若灰色地磚 A 沒有任何其他灰色地磚與其相鄰，則 A 自己形成一個灰色區塊。在本題中，請分別找出所有乾淨地磚形成的灰色及白色區塊的數量。

舉例來說，下圖所示為一個正方形區域鋪了 5x5 塊正方形地磚。污損的地磚有 4 塊，以網狀圖案表示，其餘乾淨的地磚中灰色地磚有 11 塊，形成 4 個灰色區塊，而白色地磚有 10 塊，形成 6 個白色區塊。



輸入說明：

第 1 列有 1 個數字 N，表示正方形區域中鋪了 NxN 塊正方形地磚，N 之值不超過 50。

第 2~(N+1)列共有 N 列，表示 N 列(row)地磚。同時每列有 N 個數字，表示 N 行(column)的地磚，其中數字 0 表示白色地磚，1 表示灰色地磚，而 2 則表示污損的地磚。

輸出說明：

輸出為一列，有 2 個數字，分別表示乾淨地磚所形成的灰色區塊與白色區塊之數量。

範例說明：

輸入：

```
5
1 1 0 2 0
1 2 1 0 1
2 0 1 0 1
0 1 0 2 1
1 1 0 0 0
```

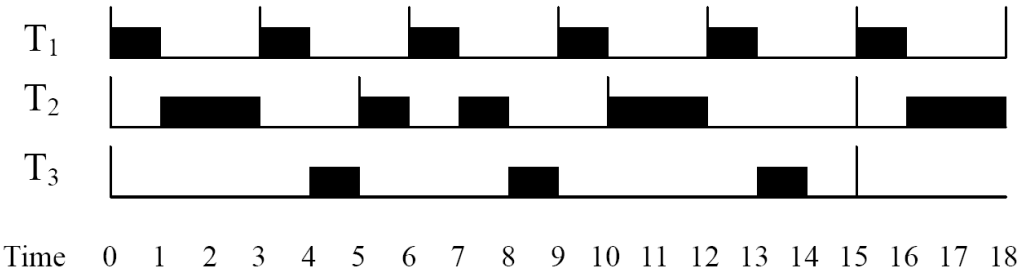
輸出：

```
4 6
```

搶佔式最早截止時間優先排程(佔分 10 分)

問題描述：

優先權驅動的排程方法通常用於現代電腦作業系統中來執行一組任務，其中系統總是執行具有最高優先權的任務。最早截止時間優先 (EDF: Earliest Deadline First) 排程方法為截止時間較早的任務分配較高的優先級，其中任務是週期性的和搶佔式(preemptive)的，我們感興趣的是所有任務至少完成一次的最早時間。



例如，在上圖中執行時間分別為 1、2、3 和週期 3、5、15 的任務 T1、T2、T3 使用搶佔式 EDF 排程器是可行的。T1、T2 和 T3 分別在時間 1、3 和 14 先完成執行，整個時間表在時間 15 重複。其中 T2 在時間 6 被 T1 搶佔並在時間 7 恢復，T3 在時間 5 被 T2 搶佔並在時間 8 恢復，在時間 9 再次被任務 T1 搶佔並在時間 13 恢復。

定義和限制

- 任務 T2 被任務 T1 搶佔，意味著優先順序較高的 T1 取代優先順序較低的 T2 來運行 CPU，T2 稍後會恢復。
- 若任務的截止時間相同，則以剩餘需運行時間較多者為優先。
- 若任務的截止時間與剩餘運行時間皆相同，以已取得 CPU 執行權者為優先。
- 就緒任務是指只要取得 CPU 使用權就可以執行的任務。
- 週期性任務在每個恆定時間間隔（稱為週期）內恰好執行一次。
- 為簡單起見，週期性任務在每個週期開始時準備就緒，其截止時間在每個週期結束時。
- 如果每個任務在截止時間之前完成執行，則此任務集是可行的。
- 所有輸入的數字均為正整數且小於或等於 500000。
- 每個任務集中的任務數量小於或等於 10。
- 每個任務每次的執行時間小於或等於其週期。
- 任務集中每個任務的週期未排序且全部不同。

輸入說明：

輸入的第一列是一個數字，表示測試案例的數量，“/”字元用來分隔兩個連續的測試案例，最後的測試案例後跟著含有“.”字元的列。對於每個測試案例，第一列給出任務數量 *n*，後面跟著 *n* 列空格分隔的任務執行時間和週期。

輸出說明：

對於每一個測試案例，將所有任務至少完成一次的最早時間列印在一列上，若該測試案例不可行，亦即存在有任務無法在週期內完成，則印出0在該列上。

範例說明：

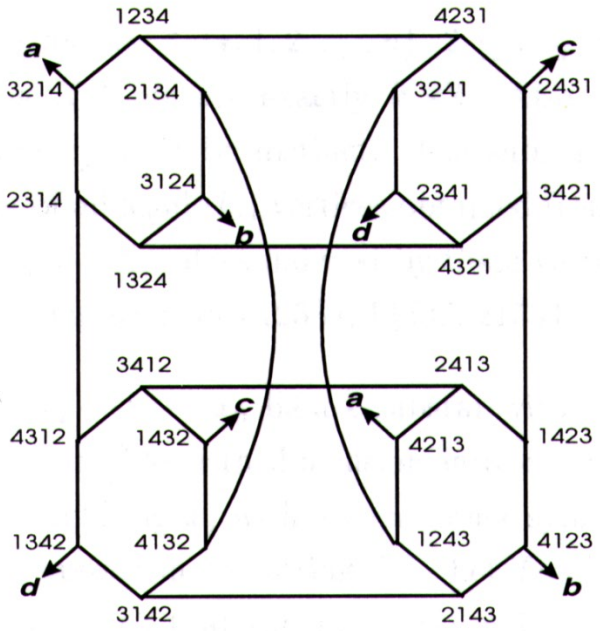
輸入：

```
3
3
1 3
2 5
3 15
/
2
1 2
1 3
/
3
1 3
2 15
2 5
.
```
輸出：
```
14
2
9
```

星圖中的最小距離(佔分 10 分)

問題描述：

在這個問題中，我們定義一個稱為星圖的圖。給定一個整數 *n*，*n* 維星圖(也稱為 S*n*)擁有 *n*！個節點和 ((*n*-1) * *n*!)/2 條邊。每個節點都被唯一分配一個標籤 $x_1\ x_2\ \ldots\ x_n$，它是 *n* 個數字 {1, 2, 3, ..., *n*} 的任意排列。例如，S_4 有以下 24 個節點 {1234, 1243, 1324, 1342, 1423, 1432, 2134, 2143, 2314, 2341, 2413, 2431, 3124, 3142, 3214, 3241, 3412, 3421, 4123, 4132, 4213, 4231, 4312, 4321}。對於每個帶有標籤 $x_1\ x_2\ x_3\ x_4\ \ldots\ x_n$ 的節點，它有 *n*-1 條邊連接到節點 $x_2\ x_1\ x_3\ x_4\ \ldots\ x_n$，$x_3\ x_2\ x_1\ x_4\ \ldots\ x_n$，$x_4\ x_2\ x_3\ x_1\ .\ .\ x_n$， ...，以及 $x_n\ x_2\ x_3\ x_4\ \ldots\ x_1$。亦即將 $x_1\ x_2\ x_3\ x_4\ \ldots\ x_n$ 的第一個符號和第 *d* 個符號交換得到 *n*-1 個相鄰節點，其中 *d*=2，...，*n*。例如，在 S_4 中，節點 1234 有 3 條邊連接到節點 2134、3214 和 4231。下圖顯示了 S_4 的樣子(注意，符號 a、b、c 和 d 不是節點，使用它們是顯示節點之間的連接性)。在這個問題上，你被要求找出星圖中兩個給定節點之間的最小距離（這是一個整數）。



輸入說明：

在這個問題中，你將獲得以下輸入：

n：星圖的維數，我們假設 *n* 的範圍是 4 到 9。

五對 S*n* 中的兩個節點標籤 $x_1\ x_2\ x_3\ x_4\ \ldots\ x_n$ 和 $y_1\ y_2\ y_3\ y_4\ \ldots\ y_n$。

輸出說明：

包含5個值的列表，每個值代表一對節點的距離。

範例說明：

輸入：

```
4
1234 4231
1234 3124
2341 1324
3214 4213
3214 2143
```

輸出：

```
1
2
2
1
3
```